

Database Systems (DV3,DAT5,SW5) Intro + SQL

Yumeng Song and Juan Manuel Rodriguez

Department of computer science, Aalborg University

Fall 2025



AALBORG
UNIVERSITY

Lecturers

That's me!



Juan Manuel Rodriguez

jmro@cs.aau.dk



Yumeng Song

yumengs@cs.aau.dk

STOP ME!

*If you have questions,
if you spot an error,
if something is different in the book,
if you are unsure about something,
if you realize you are not following,
if something is on fire,
if I start talking too fast...*



Questions

- **Asking questions** is fundamental
- I will only **answer questions** about the contents of the lectures **in class** (*in front of everyone*)
- if you do not want to “speak up” we can try to use this tool:

[`https:// bit.ly / DBSe23\(UPDATE\)`](https://bit.ly/DBSe23(UPDATE))

WHAT
DO YOU
MEAN
?

Materials

- Moodle is our source of truth:
 - [Course: Database Systems \(DBS\)\(DVML3, DAT5, AAL-SW5\)| AAU Moodle](#)
 - [General | DBS - 2025 | Microsoft Teams](#)
- Make sure you familiarize with its contents
 - slides, exercise, quiz, . . .
 - [if something is not clear, ask](#) (e.g., send an email to me and Arijit)

Textbooks

- ▶ Avi Silberschatz, Henry F. Korth, S. Sudarshan:

Database System Concepts 7th edition

McGraw-Hill -- <https://www.db-book.com/>

- ▶ Hector Garcia-Molina, Jeffrey D. Ullman, Jennifer Widom

Database Systems: The Complete Book

Slides override book

the slides of the course are self contained, always refer to the slides as the primary source for your study.

The second reference book (by Garcia-Molina et al) is offered **only as an optional alternative** for parts of the course, for students that already have it or for any other reason prefer it.

Teaching Assistants

- Asbjørn Rysgaard Eriksen – aeriks22@student.aau.dk
- Jacob Christian Harpøth Stokholm – jstokh22@student.aau.dk



TAs will be covering the various group rooms during the exercise sessions.

They are ***NOT* supposed** to be available outside time-slots of exercise sessions, so take advantage of their help during those sessions.

We have a Teams chat, find link on moodle and join!

Use it to call TAs or ask for help/questions during exercise sessions



Thanks for the slides

- ▶ Gabriela Montoya
- ▶ Katja Hose
- ▶ Matteo Lissandrini

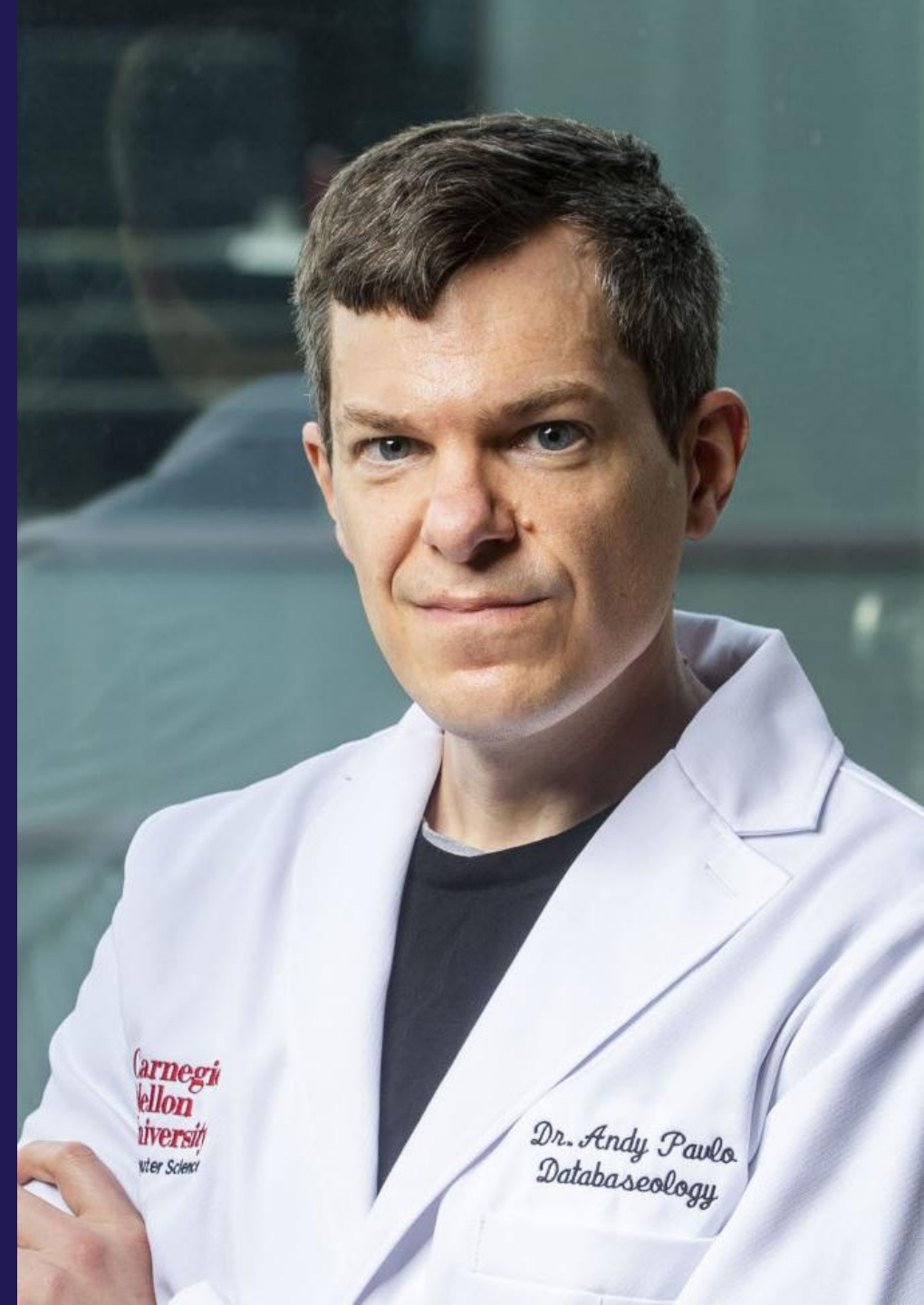
***Everything important in your
life will go through a Database***

— Andy Pavlo

<https://15445.courses.cs.cmu.edu/fall2022/>

<https://15445.courses.cs.cmu.edu/fall2023/>

*A lot of my materials is
based on his lectures*





Database vs. DBMS

What is a database?

A collection of
inter-related data that
models some aspect of the
real world

What is a database management
system (DBMS)?

A piece of software that helps
efficiently manage, manipulate, and
retrieve information from databases

<https://dbdb.io/>

Database System (DBS):
A database and a DBMS

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

★ **Core Concepts: you need to understand these**



DBMS and Data Model

- A general-purpose DBMS supports the definition, creation, querying, update, and administration of databases in accordance with some **data model**.
- A data model is **a collection of abstract concepts** for describing the data in a database: “the theory dictating the design of the database and assumed by the DBSM”
- **A schema** is a description at the **logical level** of a **particular collection of data**, using a given data model.

Where can we find databases and DBMS?

- ▶ In the cloud?..... ☒
- ▶ Datacenters?..... ☒
- ▶ In our local machines?..... ☒
- ▶ In our phones?..... ☒



Data Models

- **Relational**
- Key/Value
- Document / Object
- Wide-Column / Column-family
- Graph
- Array / Matrix / Vectors
- Hierarchical
- Network
- Multi-Value

} Legacy





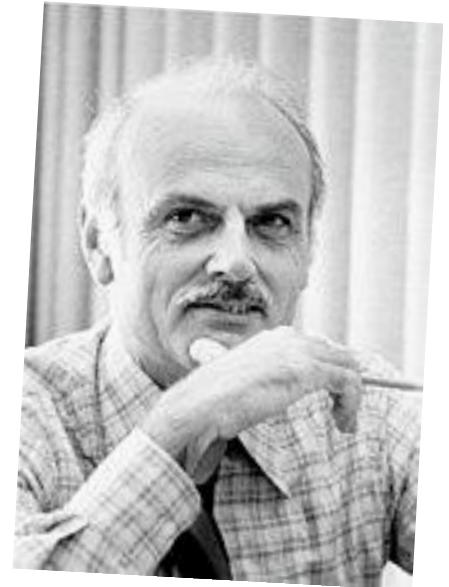
Relational Data Model intuition

The relational model defines a database abstraction based on relations to avoid maintenance overhead.

Key concepts

- Store database in **simple data structures (relations)**.
- **Physical storage** left up to the DBMS implementation.
- Access data through **high-level language**,
DBMS figures out best execution strategy.

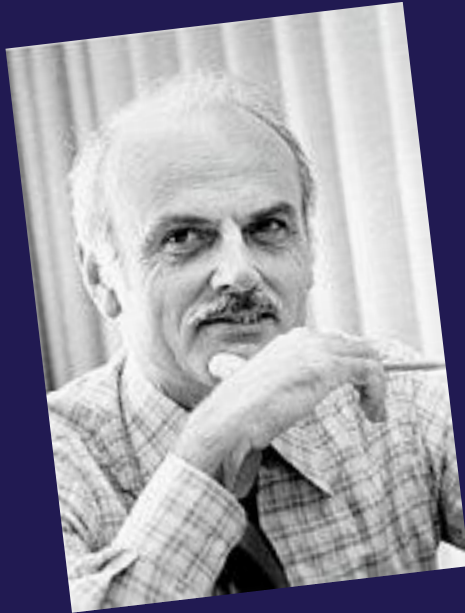
The **relational model** describes a database and the access to the database **at a logical level**
and the **DBMS** defines its implementation at the **physical level**.



Edgar F.
Codd,
1981

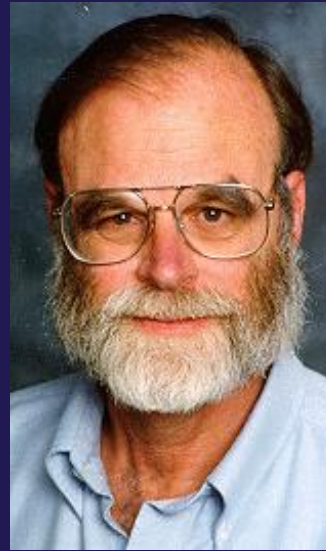
Relational Model,
Boyce-Codd normal form

Turing Award for Database Systems



Codd,
1981

**Relational Model,
Boyce–Codd normal
form**



Gray,
1998

*First implementation of
SQL,
ACID Transactions*



Stonebraker,
2014

PostgreSQL



Relational Model Concerns

The relational model defines three important concepts for a database

- **Structure:** the definition of the database's relations and their contents
- **Integrity:** the rules that ensures the database's contents satisfy application constraints
- **Manipulation:** the interface and methods for accessing and modifying a database's contents

The **relational model** describes a database and the access to the database **at a logical level**
and the **DBMS** defines its implementation at the **physical level**.

A relation

A relation is an unordered set that contains the relationships of attributes that represent entities.

A relation is usually represented as a table.

A tuple is a set of attribute values in a relation

- in the original model values are **atomic/scalar**
- It exist a special **value NULL**
- **attributes** are represented by columns in a table
- **tuples** are represented by rows in a table (sometimes called a **record**)
- a **N-ary relation** = a relation with N attributes
= a table with N-columns

course_registration(sname, ed, ctitle, ects, by)

sname	ed	ctitle	ects	by
Socrates	DV	Basics	4	Edda
Russel	DV	Basics	4	Edda
Augustinus	SW	Basics	4	Edda
Kant	DAT	Basics	4	Edda
Socrates	DV	Ethics	4	Thor
Augustinus	SW	Ethics	4	Thor
Kant	DAT	Ethics	4	Thor
Socrates	DV	Theory of Cognition	3	Locky
Russel	DV	Theory of Cognition	3	Locky
Augustinus	SW	Theory of Cognition	3	Locky

student(sname, ed, year, campus)

sname	ed	year	campus
Socrates	DV	1	AAL
Russel	DV	1	AAL
Augustinus	SW	2	CPH
Kant	DAT	2	ALL
Marx	DAT	3	CPH

cardinality {

arity }

Primary Keys

A relation's primary key uniquely identifies a single tuple

Primary Key
(sname, ctitle)



Multiple attribute can compose a primary key

By definition there cannot be 2 tuples with the same value for the primary key attribute

Primary Key
(sname)



course_registration(sname , rank, ctitle, ects, by)



sname	ed	ctitle	ects	by
Socrates	DV	Basics	4	Edda
Russel	DV	Basics	4	Edda
Augustinus	SW	Basics	4	Edda
Kant	DAT	Basics	4	Edda
Socrates	DV	Ethics	4	Thor
Augustinus	SW	Ethics	4	Thor
Kant	DAT	Ethics	4	Thor
Socrates	DV	Theory of Cognition	3	Locky
Russel	DV	Theory of Cognition	3	Locky
Augustinus	SW	Theory of Cognition	3	Locky

student(sname, ed, year, campus)

sname	ed	year	campus
Socrates	DV	1	AAL
Russel	DV	1	AAL
Augustinus	SW	2	CPH
Kant	DAT	2	ALL
Marx	DAT	3	CPH

Primary Keys

A relation's primary key uniquely identifies a single tuple

Multiple attribute can compose a primary key

New Primary Key

For practical reasons and computational optimization usually, we assign auto-generated unique integers as primary keys: **surrogate keys**

course_registration(regid, sname, ed, ctitle, ect, by)

regid	sname	ed	ctitle	ects	by
1	Socrates	DV	Basics	4	Edda
2	Russel	DV	Basics	4	Edda
3	Augustinus	SW	Basics	4	Edda
4	Kant	DAT	Basics	4	Edda
5	Socrates	DV	Ethics	4	Thor
6	Augustinus	SW	Ethics	4	Thor
7	Kant	DAT	Ethics	4	Thor
8	Socrates	DV	Theory of Cognition	3	Lucky
9	Russel	DV	Theory of Cognition	3	Lucky
10	Augustinus	SW	Theory of Cognition	3	Lucky

student(sname, ed, year, campus)

sname	ed	year	campus
Socrates	DV	1	AAL
Russel	DV	1	AAL
Augustinus	SW	2	CPH
Kant	DAT	2	ALL
Marx	DAT	3	CPH



Foreign Keys

A relation attribute that references the key of another relation

course(cid , ctitle, ects, by)

cid	ctitle	ects	by
1	Basics	4	Edda
15	Ethics	4	Thor
23	Theory of Cognition	3	Locky

course_registration(sname, cid)

sname	cid
Socrates	1
Russel	1
Augustinus	1
Kant	1
Socrates	15
Augustinus	15
Kant	15
Socrates	23
Russel	23
Augustinus	23

student(sname , ed, year, campus)

sname	ed	year	campus
Socrates	DV	1	AAL
Russel	DV	1	AAL
Augustinus	SW	2	CPH
Kant	DAT	2	ALL
Marx	DAT	3	CPH

Foreign Keys

A relation attribute that references the key of another relation

course(cid , ctitle, ects, by)

cid	ctitle	ects	by
1	Basics	4	Edda
15	Ethics	4	Thor
23	Theory of Cognition	3	Locky

course_registration(sname, cid)

sname	cid
Socrates	1
Russel	1
Augustinus	1

The Schema of a Database in the Relational Model
is defined by the set of its relation definitions, including
attributes, keys, and foreign keys



Relational Model Concerns (again)

The relational model defines three important concepts for a database

- **Structure:** the definition of the database's relations and their contents
- **Integrity:** the rules that ensures the database's contents satisfy application constraints
- **Manipulation:** the interface and methods for accessing and modifying a database's contents

The **relational model** describes a database and the access to the database **at a logical level**
and the **DBMS** defines its implementation at the **physical level**.

Data Query Language & Data Manipulation Language

There exists two types of languages

1. Procedural

→ the query specified the (high-level) sequence of operations to find the desired results

2. Declarative

→ the query specifies only what data is wanted and not how to obtain it



Data Query Language & Data Manipulation Language

There exists two types of languages

1. Procedural

→ the query specified the (high-level) sequence of operations to find the desired results

Relational Algebra

(shown later with Arijit)

2. Declarative

→ the query specifies only what data is wanted and not how to obtain it

SQL

Declarative Vs. Procedural

course(id, title, ects, taughtby)

SQL queries are declarative

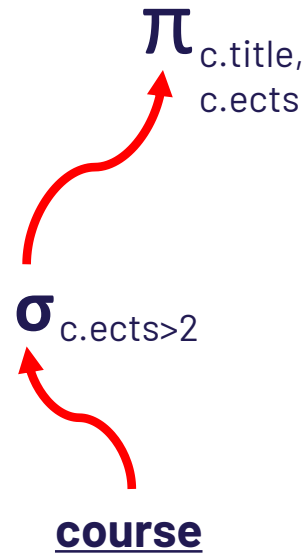
The DBMS needs to translate them into a series of operations to compute the correct result

SQL:

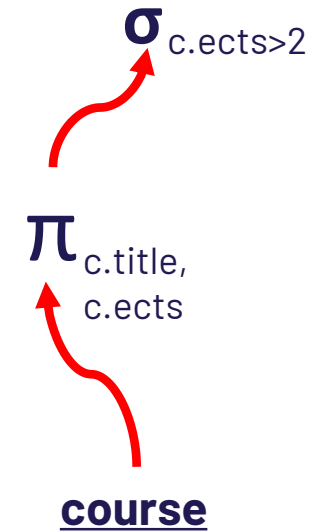
```
SELECT c.title, c.ects  
FROM course  
WHERE c.ects > 2
```

Declarative

Relational Algebra:



Procedural



Declarative Vs. Procedural

SQL queries are declarative

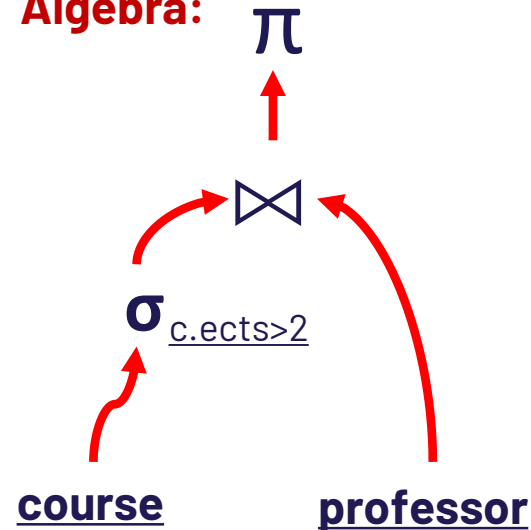
The DBMS needs to translate them into a series of operations to compute the correct result

SQL:

```
SELECT p.name, c.title, c.ects
FROM course c JOIN professor p
      ON p.id = c.taughtby
WHERE c.ects > 2;
```

Declarative

Relational Algebra:



Physical Plan:

Hash Join

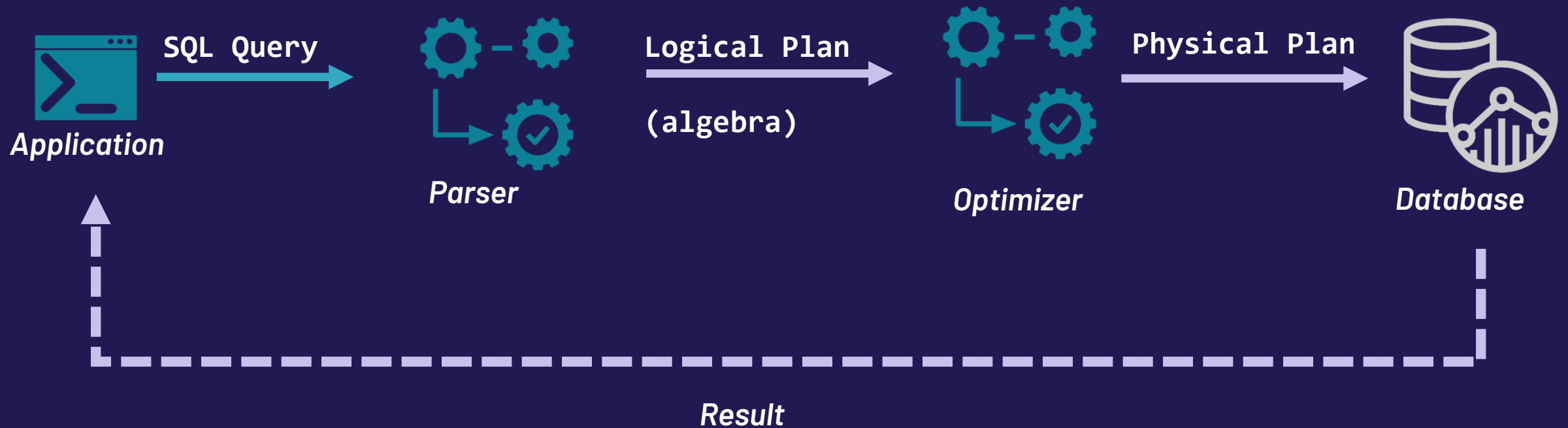
Hash Cond: (c.by = p.id)

-> Seq Scan on course c
Filter: (ects > 2)

-> Hash
-> Seq Scan on professor p

Procedural

Why it matters?



Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

SQL: *Declarative Query Language*

- ▶ You ask what you want not how to compute it
- ▶ Is not only about queries that retrieve data!
- ▶ Allows to handle all the aspects of "Database Management"
- ▶ Is different from the relational model and algebra because is based on "bags"





SETS and BAGs

- Queries in **Relational Algebra** produce only **sets** (i.e., no **duplicates**)
- **In SQL by default** queries produce **multisets** (called **bags**)

- List: [1,1,2,3,3]
- Set: { 1,2,3 }
- Bag: { { 1, 1, 2, 3, 3 } } Order doesn't matter
- Set of tuples:
{ (Kant, DV), (Kant, DAT), (Russel, DV) }
- Bag of tuples:
{ { (Kant, DV), (Kant, DV),
(Marx, DAT), (Russel, DV) } }

What SQL should we use?

ANSI Standard in 1986. ISO in 1987

→ Structured Query Language

It's an Evolving Standard

Current standard is SQL:2023

→ SQL:1999 → Regex, Triggers, OO

→ SQL:2003 → XML, Windows, Sequences, Auto-Gen IDs.

→ SQL:2008 → Truncation, Fancy Sorting

→ SQL:2011 → Temporal DBs, Pipelined DML

→ SQL:2016 → JSON, Polymorphic tables

→ SQL:2023 → Property Graph Queries, Muti-Dim. Arrays

All major database vendors (Oracle, IBM, Microsoft) conform to some portion of SQL standard

...but most system implement a dialect, so some keywords differ, and behaviours are different!

In doubt refer to Postgres



The minimum language syntax a system needs to say that it supports SQL is **SQL-92**.



SQL – Not only SELECT queries

Data Definition
Language

CREATE TABLE

DDL

Data Manipulation
Language

INSERT INTO
DELETE FROM
UPDATE

DML

Data Query
Language

SELECT FROM
WHERE

DQL

Transaction
Control Language

BEGIN;

COMMIT;

ROLLBACK;

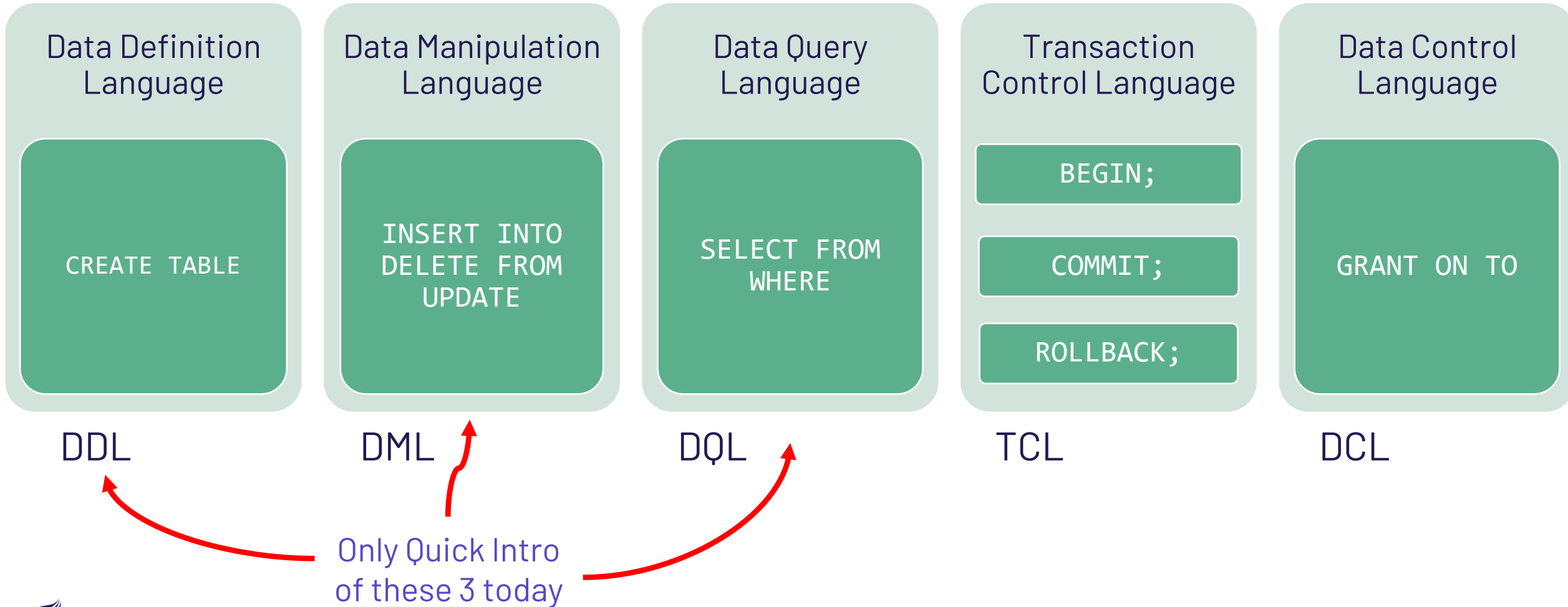
TCL

Data Control
Language

GRANT ON TO

DCL

SQL – Not only SELECT queries



Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

★ **Core Concepts: you need to understand these**



Data Definition Language: CREATE

To make a (relational) database you need a table

```
CREATE TABLE professor(  
    id integer    UNIQUE NOT NULL ,  
    name varchar(10)    NOT NULL ,  
    rank char(2)  
);
```

Types for Numbers:

INT, BIGINT,
SMALLINT,
FLOAT

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

- char(n) always stores n characters
- varchar(n) uses only the required chars, plus length information
- type "TEXT" is not in the standard but is often the best option!

UNIQUE & NOT NULL are
called constraints
(see next slides)

varchar(n) vs. char(n)

- Both varchar(n) and char(n) are limited to a length of n
- char(n) always uses n bytes
- varchar(n) uses only the required place, plus length information

<https://www.postgresql.org/docs/current/datatype-character.html>

- when you insert less than n characters in an attribute declared char(n) operations on strings (e.g., equality) can behave unexpectedly, **test your system!**

CREATE TABLE AS

```
CREATE TABLE professor(  
    id integer UNIQUE NOT NULL,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

***It will not carry on
the constraints!***

In postgres use the commands
`\d+ professor`
`\d+ profs`
to see the differences

```
CREATE TABLE profs AS  
    (SELECT * FROM professor);
```

profs

id	name	rank
----	------	------

*PostgreSQL (does not follow the SQL:2003
standard and) populates the table by default*

```
CREATE TABLE profs2 AS  
    (SELECT * FROM professor)  
WITH DATA;
```

profs2

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

Syntax & Case Sensitivity

- SQL has reserved keywords, e.g., CREATE, SELECT, TABLE, WHERE, PRIMARY
- The reserved keywords are case-insensitive: you can write both **create** and **CREATE**, the command is the same
- reserved keywords cannot be used for table names or attribute names
- when we use strings, we use single quotes 'Mette', 'A.L.', '@cs.aau.dk', these strings are case-sensitive (in the standard, implementation may differ)
- *All commands/statements/queries need to end with a semicolon (;)*
- *indentation and white space does not matter*



KEYS & other constraints

- UNIQUE declares an attribute **cannot contain multiple copies** of the same value
- PRIMARY KEY declares an attribute as **primary key** → only one primary key per table, can be composite!
- The primary key constraint **includes** the NOT NULL and UNIQUE constraints.

```
CREATE TABLE professor(  
    id integer PRIMARY KEY ,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);
```

```
CREATE TABLE advisor(  
    studentID integer UNIQUE,  
    professorID integer,  
    project varchar(140) NOT NULL,  
    PRIMARY KEY(studentID, professorID)  
);
```

```
CREATE TABLE professor(  
    id integer,  
    name varchar(10) NOT NULL,  
    rank char(2),  
    PRIMARY KEY(id)  
);
```

**Correct, but what's
going on here!?!**

Static integrity constraints

- Enforce constraints based on the current status of the table and the values
- Are enforced for every data manipulation operation

```
CREATE TABLE professor(  
    id integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2),  
    CHECK rank IN ('C2', 'C3', 'C4')  
);
```

```
CREATE TABLE student (  
    studid integer PRIMARY KEY,  
    name varchar (30) NOT NULL,  
    semester integer,  
    CHECK semester BETWEEN 1 AND 20  
);
```




DEFAULT Values

When inserting data into a table, all values that are not explicitly stated are set to **NULL** (standard default value).

When defining a table, we can specify another default value.

Default values can be changed later by other operations!

```
CREATE TABLE professor(  
    id integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2) DEFAULT 'C1'  
);
```

SEQUENCE Number Generators

Sequence number generators automatically create continuous identifiers.

Can be used for automatic **surrogate keys** generation

```
CREATE SEQUENCE professorseq START 2100;
```

```
CREATE TABLE professor(  
    id integer PRIMARY KEY DEFAULT NEXT VALUE FOR professorseq,  
    name varchar(10) NOT NULL,  
    rank char(2) DEFAULT 'C1'  
);
```

In PostgreSQL:
DEFAULT nextval('professorseq')



FOREIGN KEYS (I)

Valid values of a foreign key **must reference an existing value of a candidate key** in the referenced table

```
CREATE TABLE professor(  
    id integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);
```

```
CREATE TABLE course(  
    courseid integer PRIMARY KEY,  
    title varchar(30) NOT NULL,  
    ects integer,  
    taughtby integer,  
    FOREIGN KEY(taughtby) REFERENCES professor(id)  
);
```

```
CREATE TABLE advisor(  
    studentID integer UNIQUE,  
    professorID integer,  
    project varchar(140) NOT NULL,  
    PRIMARY KEY(studentID, professorID),  
    FOREIGN KEY(professorID) REFERENCES professor(id),  
    FOREIGN KEY(studentID) REFERENCES student(id)  
);
```



FOREIGN KEYS (II)

Foreign keys can also reference candidate keys enforced with **UNIQUE** constraints

```
CREATE TABLE course(  
    courseid integer PRIMARY KEY,  
    title varchar(30) UNIQUE NOT NULL,  
    credits integer  
);
```

```
CREATE TABLE teaches(  
    title varchar(30) REFERENCES course(title),  
    pid integer REFERENCES professor(id),  
    semester char(1)  
);
```

```
CREATE TABLE professor(  
    id integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);
```

In **relational schema** you can use any attribute that makes logical sense as keys. In a DBMS, using text as keys is generally bad practice for performance reasons!
Use integer surrogate keys when possible!





Changing Table definitions

After creating a table, it is possible to add/remove attributes and constraints

```
CREATE TABLE professor(  
    id integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2) DEFAULT 'C1'  
);
```



You can also drop constraints!
*Not part of the exam,
left to the student's curiosity.*

Add an attribute

```
ALTER TABLE professor  
ADD COLUMN office integer;
```

Delete an attribute

```
ALTER TABLE professor  
DROP COLUMN rank;
```

Change an attribute type

```
ALTER TABLE professor  
ALTER COLUMN name type varchar(30);
```

Add an attribute constraint

```
ALTER TABLE professor  
ALTER COLUMN office SET NOT NULL;
```



"DELETING"...

- Delete a table

DROP TABLE professor;

- Delete the content of a table (not the table!)

TRUNCATE TABLE professor;

- TRUNCATE **cannot be used if there is another table with a foreign key** referencing the table we want to delete...
- The TRUNCATE TABLE statement is logically (though not physically) equivalent to the **DELETE FROM** statement (without a WHERE clause) – we will see this later.
- TRUNCATE is in DDL DELETE is in DML!! ← this matters! we will see why...
- What about DROP COLUMN? Same as TRUNCATE!

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

★ **Core Concepts: you need to understand these**

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.



DML: Data Manipulation Language, INSERT (I)

```
CREATE TABLE professor(  
    id integer PRIMARY KEY,  
    name varchar(10) NOT NULL,  
    rank char(2) DEFAULT 'C1'  
);
```

```
INSERT INTO professor VALUES  
    (2136, 'Curie', 'C4');
```

```
INSERT INTO professor(name, id) VALUES  
    ('Curie', 2146);
```

```
INSERT INTO professor VALUES  
    (2156, 'Christian');
```

```
INSERT INTO professor VALUES  
    (2156, 'Immanuel');
```

Will be denied!

If you assign a sequence to a primary key,
then do not insert IDs yourself... what happens if you try?



id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2136	Curie	C4
2146	Curie	C1
2156	Christian	C1



DML: DELETE (I)

```
CREATE TABLE student (  
    studid integer UNIQUE NOT NULL,  
    name varchar(30) NOT NULL,  
    semester integer  
);
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

DELETE FROM student;



student

studid	name	semester
--------	------	----------



DML: DELETE (II)

```
CREATE TABLE student (  
    studid integer UNIQUE NOT NULL,  
    name varchar(30) NOT NULL,  
    semester integer  
);
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

DELETE FROM student
WHERE semester > 13;



student

studid	name	semester
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Outline

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples / Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

★ **Core Concepts: you need to understand these**

SQL Dialects vs. SQL Standards

Dialects [1]

There is an SQL standards, but different vendors [i.e., software] accept different dialects, so they have keywords/commands that work a bit different or have more options!

- (1) In this course (exam) stick to the keywords and functionalities we present**
- (2) PostgreSQL follows most (not all) the standard**



SQL Standards [2]

There are several versions of the SQL standard (SQL-86, SQL-89, SQL-92, SQL:1999, SQL:2003, SQL:2011, SQL:2016), each one adds keywords, options, and functionalities, this course focuses around SQL-92

[1] https://en.wikibooks.org/wiki/SQL_Dialects_Reference

[2] <https://modern-sql.com/standard>



Data Query Language

SFW: *The Basic Building Blocks of an SQL query*

SELECT [list of columns]

FROM [list of tables]

WHERE [conditions]

- Called “projection” list, it can include arithmetic operations, it can include *nested queries*
- Table list, can involve temporary tables, and renaming, it can include *nested queries*
- Selection conditions, Join conditions, it can include *nested queries*



The projection columns

```
SELECT [DISTINCT] projectionList  
FROM ...
```

- It can include a list of:
 - **Columns of tables in the FROM clause**
 - **Arithmetic expressions** over constants and columns of tables in the FROM clause
 - **Aggregate functions** over columns of tables in the FROM clause (see next lecture)
 - the symbol "*" to represents all columns of all tables in the FROM clause



Example

Every SQL query needs to be terminated by a semicolon (;)



professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

SELECT *
FROM professor ;



id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4



Example (II) + DISTINCT

SELECT name
FROM professor;

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

SELECT **DISTINCT** name
FROM professor;

name
Socrates
Russel
Russel
Popper
Augustinus
Kant

The results is a **bag** (multiset)

*Output Order
can be arbitrary*



name
Kant
Russel
Popper
Augustinus
Socrates

The results is a **set**
and corresponds
to the projection
operation in
relational algebra



Example (III) + renaming

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

SELECT (3+5)
FROM professor;

??
8
8
8
8
8
8

SELECT (2*5) **AS** foo
FROM professor;

foo
10
10
10
10
10
10

The keyword AS is optional

SELECT name n, (id+5) foo
FROM professor;



Conditions on a single table

```
SELECT id, name  
FROM professor  
WHERE rank = 'C4';
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2135	Marx	c4
2134	Augustinus	C3
2137	Kant	C4



id	name
2125	Socrates
2126	Russel
2137	Kant



Conditions on a single table (II)

```
SELECT name, id  
FROM professor  
WHERE ( rank = 'C4' OR rank = 'c4' ) AND id < 2137;
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2135	Marx	c4
2134	Augustinus	C3
2137	Kant	C4



name	id
Socrates	2125
Russel	2126
Marx	2135



Conditions

```
SELECT ...  
FROM ...  
WHERE condition
```

- Comparisons to other columns and/or constants
 - Operators: =, **<>**, >, <, >=, <=, LIKE, BETWEEN, IN, IS
- Combinations of conditions using operators AND, OR, NOT

<> means not equal

IS NULL tests for null values

BETWEEN

- BETWEEN is used for ranges
- Students in the first four semesters

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	4
29120	Theophrastos	2
29555	Feuerbach	1

```
SELECT studid, name
FROM student
WHERE semester BETWEEN 1 AND 4;
```

studid	name
28106	Carnap
29120	Theophrastos
29555	Feuerbach



LIKE

- LIKE is used for regular expressions
- Students with names that start with "F"

"%" represents an arbitrary string (including length 0)
There is also "_" which represents a single (not empty) arbitrary character

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2
30000	Fe	3

```
SELECT studid, name  
FROM student  
WHERE name LIKE 'F%';
```

studid	name
26120	Fichte
29555	Feuerbach
30000	Fe

```
SELECT studid, name  
FROM student  
WHERE name LIKE 'F_';
```

studid	name
30000	Fe



Cross Product (Cartesian product)

- If the **FROM** clause enumerates **more than a single table**, the **cross product is computed – a.k.a. the cartesian product**
- The result is the set of **all combinations of tuples** in the involved tables

```
SELECT *  
FROM professor, course;
```

```
SELECT *  
FROM professor CROSS JOIN course;
```

↑
Equivalent!



Cross Product (Cartesian product)

```
SELECT *  
FROM professor, course;
```

professor

name	pid
Socrates	2125
Russel	2126

course

title	cid
Basics	945
Ethics	946



name	pid	title	cid
Socrates	2125	Ethics	946
Russel	2126	Ethics	946
Socrates	2125	Basics	945
Russel	2126	Basics	945



Cross Product – attribute names

```
SELECT *  
FROM professor, course;
```

professor

name	id
Socrates	2125
Russel	2126

course

title	id
Basics	945
Ethics	946



name	id	title	id
Socrates	2125	Ethics	946
Russel	2126	Ethics	946
Socrates	2125	Basics	945
Russel	2126	Basics	945

```
SELECT id  
FROM professor, course;
```

ERROR: column reference "id" is ambiguous

```
SELECT course.id  
FROM professor, course;
```

id
946
946
945
945



Cross Product – attribute names

```
SELECT *  
FROM professor, course;
```

professor(name, id)
course(id, title, taughtby → professor)

professor

name	id
Socrates	2125
Russel	2126

course

title	id	taughtby
Basics	945	2125
Ethics	946	2130



name	id	title	id	taughtby
Socrates	2125	Ethics	946	2130
Russel	2126	Ethics	946	2130
Socrates	2125	Basics	945	2125
Russel	2126	Basics	945	2125

 Schema!

 Foreign
keys have
no effect
on SQL
SELECT
queries!



JOIN (theta join)

Common **Join** is expressed by **cross product** in combination with a **predicate** in the WHERE clause

professor(name, id)
course(id, title, taughtby→professor)

Schema!

```
SELECT name, title
FROM professor, course
WHERE professor.id=course.taughtby;
```

professor

name	id
Socrates	2125
Russet	2126

course

id	title	taughtby
945	Basics	2125
946	Ethics	2125
950	Morals	2130



name	title
Socrates	Basics
Socrates	Ethics

Equivalent!

```
SELECT name, title
FROM professor JOIN course
ON professor.id=course.taughtby;
```



Common JOIN (self join)

professor(id, name, rank)

```
SELECT p1.name, p2.name
FROM professor AS p1 JOIN professor AS p2
      ON p1.rank = p2.rank+1
```

professor (p1)

id	name	rank
2125	Socrates	1
2126	Russel	2
2130	Marx	3
2127	Plato	3

professor (p2)

id	name	rank
2125	Socrates	1
2126	Russel	2
2130	Marx	3
2127	Plato	3



name	name
Socrates	Russel
Russel	Plato
Russel	Marx

Here we apply renaming
of tables in the query
(we use keyword **AS** but is optional)

Naming intermediate results

- ▶ Intermediate result tables obtained from SQL operations or SFW blocks can be assigned names (tuple variables)

```
SELECT teaching.name  
FROM (professor NATURAL JOIN course) AS teaching;
```

The keyword AS is optional

- ▶ Will be very useful with advanced query constructs (next lecture)



DML: INSERT (II)

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8

course

courseid	title	ects	taughtBy
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

You can insert in a table the
result of a query

```
CREATE TABLE takes(  
  studid integer references student(studid),  
  courseid integer references course(courseid)  
);
```

```
INSERT INTO takes  
  SELECT studid, courseid  
  FROM student, course  
  WHERE title = 'Basics';
```

What is this doing now?

takes

studid	courseid
24002	5001
25403	5001
26120	5001
26830	5001



Natural JOIN

Natural Join will join tables with common attributes
(*names must be the same*)

professor

name	pid
Socrates	2125
Russel	2126



desk

deskid	room	pid
D45	rm0.4	2125
D46	rm1.8	2125
D50	rm0.4	2130

pid	name	room
2125	Socrates	rm0.4
2125	Socrates	rm1.8

professor(name, pid)
desk(deskid, room, pid → professor)

```
SELECT pid, name, room  
FROM professor NATURAL JOIN desk
```

```
SELECT pid, name, room  
FROM professor JOIN desk USING(pid)
```

NATURAL JOIN will output the join attribute only once because is "known" to be the same





Natural JOIN (II)

professor(name, id)
course(id, title, taughtby→professor)

professor

name	id
Socrates	2125
Russel	2126



course

id	title	taughtby
945	Basics	2125
946	Ethics	2125
950	Morals	2130

name	title
------	-------



No values in
common!



```
SELECT name, title  
FROM professor NATURAL JOIN course;
```




Natural JOIN (III)

professor(name, id)
course(cid, title, taughtby→professor)

professor

name	id
Socrates	2125
Russel	2126



course

cid	title	taughtby
945	Basics	2125
946	Ethics	2125
950	Morals	2130

name	title
Socrates	Basics
Russel	Basics
Socrates	Ethics
Russel	Ethics
Socrates	Morals
Russel	Morals

```
SELECT name, title  
FROM professor NATURAL JOIN course;
```

When no attributes are in
common NATURAL JOIN
behaves as cross product





Set Operations

professor

id	name	rank
2101	Christian	C4
2102	Jens	C4
2103	Kristian	C3
2104	Christian	C4

student

studid	name	semester
97001	Jens	9
97002	Martin	7
97003	Helle	5
97004	Christian	1

- Set operations require **union compatibility**
 - Same number of attributes in the correct order
 - Compatible domains: identical domains, domains based on characters, or domains based on numerical values
- Result schema: column names of the **first** table

```
SELECT id, name FROM professor
UNION
SELECT studid, name FROM student;
```

	id	name
professor	2101	Christian
	2102	Jens
	2103	Kristian
	2104	Christian
student	97001	Jens
	97002	Martin
	97003	Helle
	97004	Christian



Set Operations

professor

id	name	rank
2101	Christian	C4
2102	Jens	C4
2103	Kristian	C3
2104	Christian	C4

student

studid	name	semester
97001	Jens	9
97002	Martin	7
97003	Helle	5
97004	Christian	1

- Set operations require **union compatibility**
 - Same number of attributes in the correct order
 - Compatible domains:

```
SELECT id, name, rank FROM professor
UNION
SELECT studid, name FROM student;
```

✗ wrong

wrong ✗

```
SELECT name, id FROM professor
UNION
SELECT studid, name FROM student;
```



Duplicates and set operations

professor

id	name	rank
2101	Christian	C4
2102	Jens	C4
2103	Kristian	C3
2104	Christian	C4

student

studid	name	semester
27001	Jens	9
27002	Martin	7
27003	Helle	5
27004	Christian	1

For set operations, **duplicate elimination** (e.g., UNION **DISTINCT**) is the default

(SELECT name FROM professor)

UNION

(SELECT name FROM student);

name
Christian
Jens
Kristian
Martin
Helle

(SELECT name FROM professor)

UNION ALL

(SELECT name FROM student);

	name
professor	Christian
	Jens
	Kristian
	Christian
student	Jens
	Martin
	Helle
	Christian



Intersect and Except

A

x
1
2
3

B

x
2
3
4

(SELECT * FROM A)
INTERSECT
(SELECT * FROM B);

x
2
3

(SELECT * FROM A)
EXCEPT
(SELECT * FROM B);

x
1

INTERSECT and EXCEPT can also be combined with ALL (analogous to UNION)

Set operators in combination with ALL are not supported by all DBMSs



Ordering

```
SELECT name  
FROM professor  
ORDER BY rank;
```

```
SELECT name  
FROM professor  
ORDER BY rank ASC;
```

ASC: ascending
DESC: descending

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

	name
C2	Russel
C3	Popper
	Augustinus
C4	Socrates
	Russel
	Kant

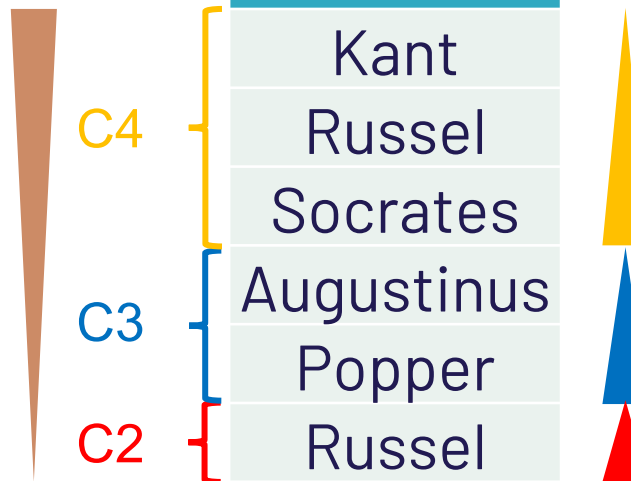
The output tuples are sorted ascending by default

Ordering multiple fields

```
SELECT name  
FROM professor  
ORDER BY rank DESC, name ASC;
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4



ASC: ascending
DESC: descending

Summary & Preview of SQL

- ▶ The relational model works at the LOGICAL level
- ▶ SQL is more than a query language (DDL, DML, DCL, TCL, DQL)
- ▶ DQL is a declarative query language, SFW block is the basis
- ▶ The JOIN is the core of SQL
- ▶ Next
 - Complex JOINS, nested queries, NULLs!
 - Grouping and aggregation
 - Recursive queries
 - Updates & Handling Integrity constraints
 - Views



Best way to learn is to try!

- Find the exercises on Moodle
- Try to solve them
- Install Postgres (some help is on moodle), try to recreate examples and exercises
- You can use online tools also
<https://extendsclass.com/postgresql-online.html>

